

TUGAS 6 – DEADLOCK, PENCEGAHAN DEADLOCK DAN BANKERS ALGORITHM

SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyanto, M.K

Asisten Dosen :
Atidhira Habibillah dan Taqiyyuddin

Disusun Oleh:
Muhammad Syarifuddin (2210651057)

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

DAFTAR ISI

TUGAS 6 – DEADLOCK, PENCEGAHAN DEADLOCK DAN BANKERS ALGORITHM.....	1
DAFTAR ISI.....	2
1. TUJUAN PRAKTIKUM.....	3
2. DASAR TEORI.....	4
3. Tugas 1 — ANALISIS DEADLOCK.....	5
4. Tugas 2 — DEADLOCK PREVENTION.....	8
5. Tugas 3 — BANKER’S ALGORITHM.....	8
6. Tugas 4 — SIMULASI DEADLOCK TRANSFER BANK.....	8
7. KESIMPULAN.....	9

1. TUJUAN PRAKTIKUM

- Memahami bagaimana deadlock dapat terjadi pada program multithreading.
- Menganalisis kondisi yang menyebabkan deadlock (4 kondisi Coffman).
- Menerapkan dua teknik pencegahan deadlock:
 - a. Lock ordering
 - b. Try-lock dengan retry
- Menerapkan algoritma Banker's Algorithm untuk mengecek kondisi aman sistem.
- Mensimulasikan kasus deadlock nyata (transfer bank dua arah) dan solusinya.

2. DASAR TEORI

a. Deadlock

Deadlock adalah kondisi ketika dua atau lebih proses saling menunggu sehingga tidak ada yang bisa melanjutkan eksekusi. Deadlock biasanya terjadi pada sistem yang menggunakan resource yang harus dikunci (mutex, semaphore).

b. Empat kondisi deadlock (coffman conditions)

Deadlock hanya bisa terjadi ketika keempat syarat berikut terpenuhi:

1. **Mutual Exclusion** – resource tidak dapat digunakan lebih dari satu proses pada saat yang sama.
2. **Hold and Wait** – proses memegang satu resource sembari menunggu resource lain.
3. **No Preemption** – resource yang sedang digunakan tidak bisa diambil paksa.
4. **Circular Wait** – terdapat rantai proses yang saling menunggu resource satu sama lain.

c. Pencegahan deadlock

Teknik umum untuk mencegah deadlock:

1. **Lock Ordering** — Semua thread mengunci resource dalam urutan yang sama.
2. **Try-lock (non-blocking)** -- Menggunakan `pthread_mutex_trylock` dan mencoba ulang (retry) bila gagal.

d. Banker's algorithm

Digunakan untuk menentukan apakah sistem berada pada keadaan aman (safe state) sebelum memberikan resource tambahan kepada proses.

3. TUGAS 1 — ANALISIS DEADLOCK

- Buatlah file dengan nama deadlock_simple.c dan isi dengan kode berikut

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA;
pthread_mutex_t lockB;

void *thread1(void *arg) {
    pthread_mutex_lock(&lockA);
    printf("Thread 1: mengunci A\n");
    sleep(1);

    pthread_mutex_lock(&lockB);
    printf("Thread 1: mengunci B\n");

    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lockB);
    printf("Thread 2: mengunci B\n");
    sleep(1);

    pthread_mutex_lock(&lockA);
    printf("Thread 2: mengunci A\n");

    pthread_mutex_unlock(&lockA);
    pthread_mutex_unlock(&lockB);
    return NULL;
}
```

```

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

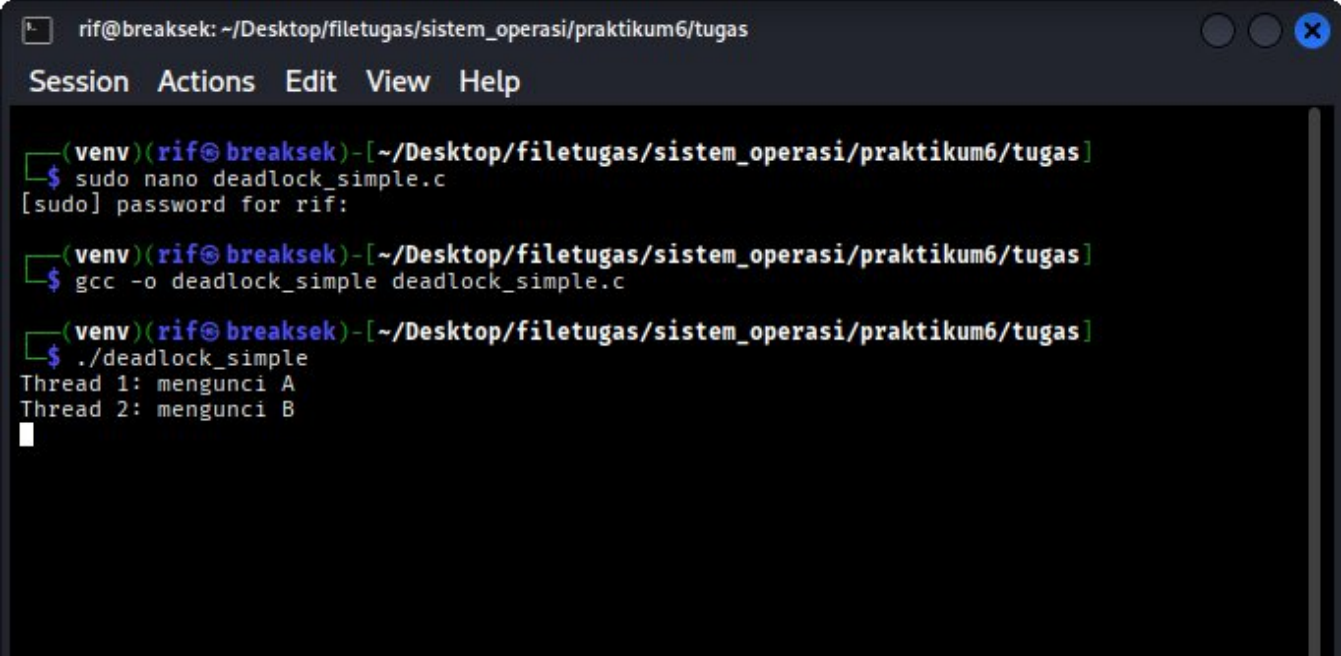
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

- Compile dan jalankan program



The screenshot shows a terminal window with the following commands and output:

```

rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas
Session Actions Edit View Help

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano deadlock_simple.c
[sudo] password for rif:

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o deadlock_simple deadlock_simple.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./deadlock_simple
Thread 1: mengunci A
Thread 2: mengunci B

```

- **Analisa :**
 1. **Mengapa program mengalami deadlock?**

Deadlock muncul karena kedua thread mengunci resource secara silang:

- Thread 1 mengunci A lalu menunggu B

- Thread 2 mengunci B lalu menunggu A

Keduanya saling menunggu sehingga tidak ada yang bisa melanjutkan.

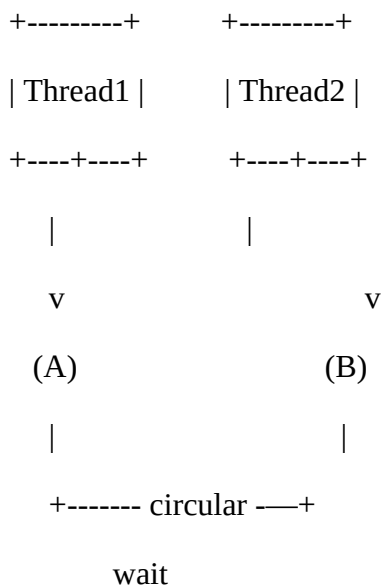
2. Kondisi deadlock apa saja yang terpenuhi?

Kondisi Coffman	Status	Penjelasan
Mutual Exclusion	✓	Mutex hanya dapat dipegang 1 thread
Hold and Wait	✓	T1 memegang A menunggu B, T2 memegang B menunggu A
No Preemption	✓	Mutex tidak bisa dipaksa dilepas
Circular Wait	✓	T1 menunggu B → T2 menunggu A → lingkaran

3. Resource Allocation Graph

Thread1 → A → Thread2 → B → Thread1

Diagram :



4. TUGAS 2 — IMPLEMENTASI PENCEGAHAN

1. Lock Ordering

Setiap thread mengunci mutex dengan urutan yang sama → tidak ada circular wait.

1.1. Buat file dengan nama `deadlock_lock_ordering.c` dan masukkan kode berikut

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lockA, lockB;

void lock_in_order(pthread_mutex_t *x, pthread_mutex_t *y) {
    if (x < y) {
        pthread_mutex_lock(x);
        pthread_mutex_lock(y);
    } else {
        pthread_mutex_lock(y);
        pthread_mutex_lock(x);
    }
}

void unlock_in_order(pthread_mutex_t *x, pthread_mutex_t *y) {
    if (x < y) {
        pthread_mutex_unlock(y);
        pthread_mutex_unlock(x);
    } else {
        pthread_mutex_unlock(x);
        pthread_mutex_unlock(y);
    }
}

void *thread_func(void *arg) {
    lock_in_order(&lockA, &lockB);
```



```

printf("Thread: kerja selesai tanpa deadlock\n");
unlock_in_order(&lockA, &lockB);
}

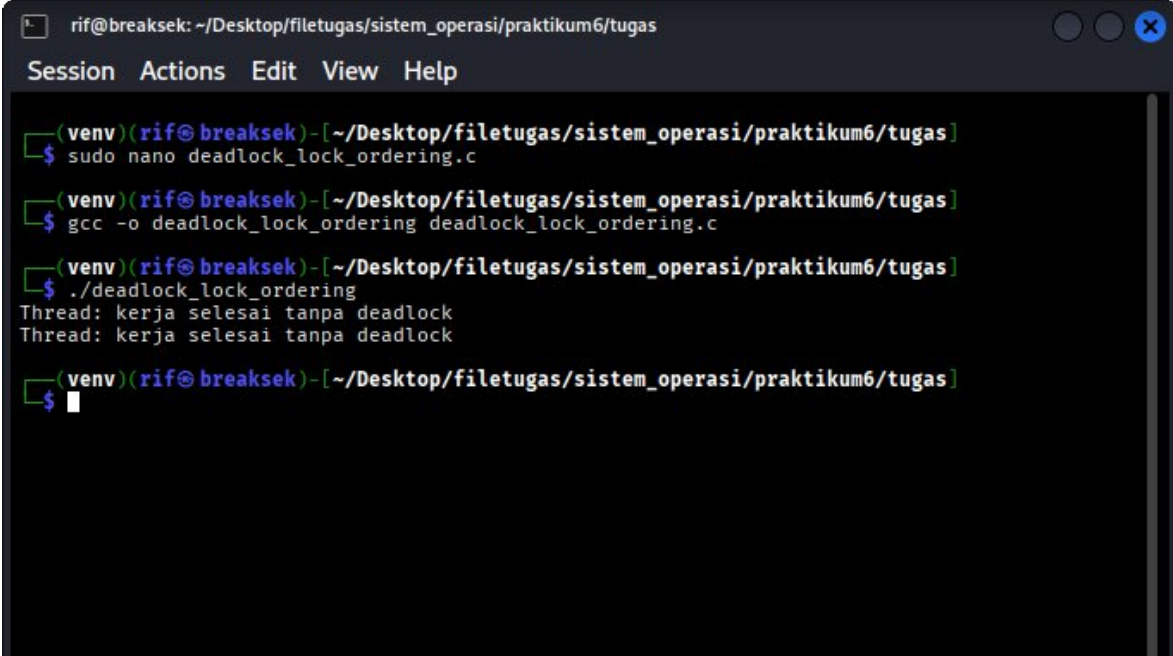
int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}

```

1.2. Compile dan jalankan program



```

rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas
Session Actions Edit View Help

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano deadlock_lock_ordering.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o deadlock_lock_ordering deadlock_lock_ordering.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./deadlock_lock_ordering
Thread: kerja selesai tanpa deadlock
Thread: kerja selesai tanpa deadlock

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$

```

2. Try-lock dengan retry

Thread mencoba mengunci mutex. Jika gagal → lepaskan semua lock dan ulangi.

2.1. buat file dengan nama deadlock_trylock.c dan masukkan kode berikut

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA, lockB;

void *thread_func(void *arg) {
    while (1) {
        if (pthread_mutex_trylock(&lockA) == 0) {

            if (pthread_mutex_trylock(&lockB) == 0) {
                printf("Thread berhasil mendapat A & B\n");

                pthread_mutex_unlock(&lockB);
                pthread_mutex_unlock(&lockA);
                break;
            } else {
                pthread_mutex_unlock(&lockA);
                usleep(100000);
            }
        }
    }
    return NULL;
}

int main() {
    pthread_t t;

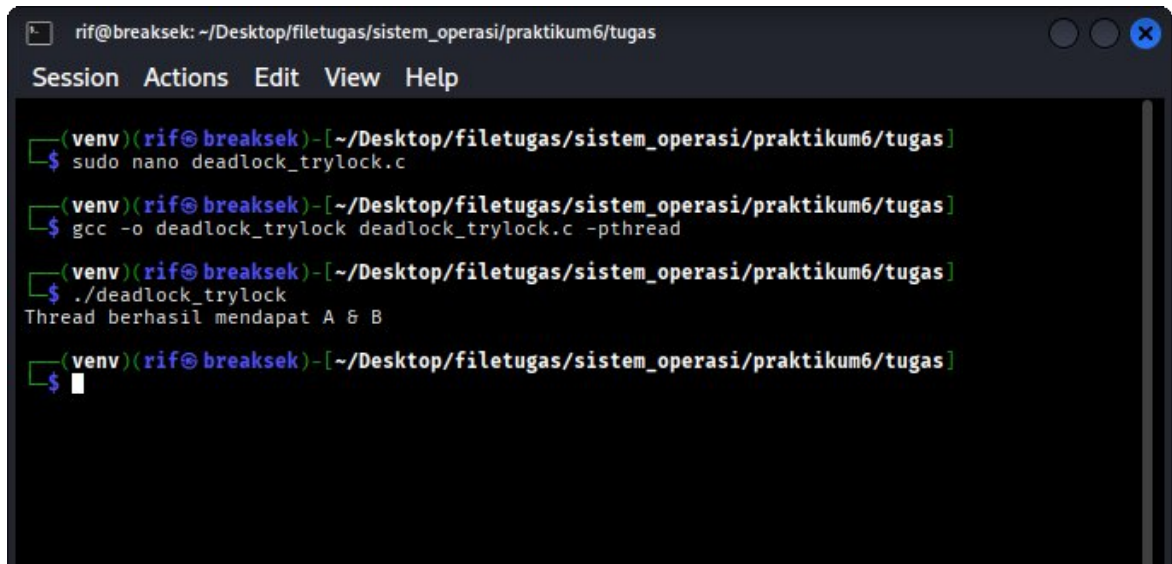
    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t, NULL, thread_func, NULL);
    pthread_join(t, NULL);
}
```

```
pthread_mutex_destroy(&lockA);
pthread_mutex_destroy(&lockB);

return 0;
}
```

2.2. Compile dan jalankan program



```
rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas
Session Actions Edit View Help

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano deadlock_trylock.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o deadlock_trylock deadlock_trylock.c -pthread

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./deadlock_trylock
Thread berhasil mendapat A & B

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$
```

3. Perbandingan kedua metode

Aspek	Lock Ordering	Try-lock
Risiko deadlock	Sangat rendah	Tidak ada
Overhead CPU	Rendah	Lebih tinggi (busy retry)
Kemudahan implementasi	Perlu merancang urutan	Lebih fleksibel
Performa	Stabil	Bisa fluktuatif

4. Kesimpulan

Lock ordering lebih efisien, sedangkan try-lock lebih fleksibel untuk kasus resource dinamis.

5. TUGAS 3 — BANKER'S ALGORITHM

- buat file dengan nama banker.c dan masukkan kode berikut

```
#include <stdio.h>

int n = 4;
int m = 3;

int available[3] = {5,4,3};

int max[4][3] = {
    {3,2,2},
    {4,1,1},
    {2,2,2},
    {4,3,1}
};

int alloc[4][3] = {
    {1,0,0},
    {2,1,0},
    {1,1,1},
    {0,0,1}
};

int need[4][3];

void calc_need(){
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];
}

int is_safe() {
    int work[3];
    int finish[4]={0};
```

```

for(int i=0;i<m;i++)
    work[i] = available[i];

int done=0;
while(done<n){
    int found = 0;

    for(int i=0;i<n;i++){
        if(!finish[i]){
            int ok = 1;

            for(int j=0;j<m;j++){
                if(need[i][j] > work[j]){
                    ok=0;
                    break;
                }
            }

            if(ok){
                for(int k=0;k<m;k++){
                    work[k]+=alloc[i][k];

                    finish[i]=1;
                    found=1;
                    done++;
                }
            }
        }
    }

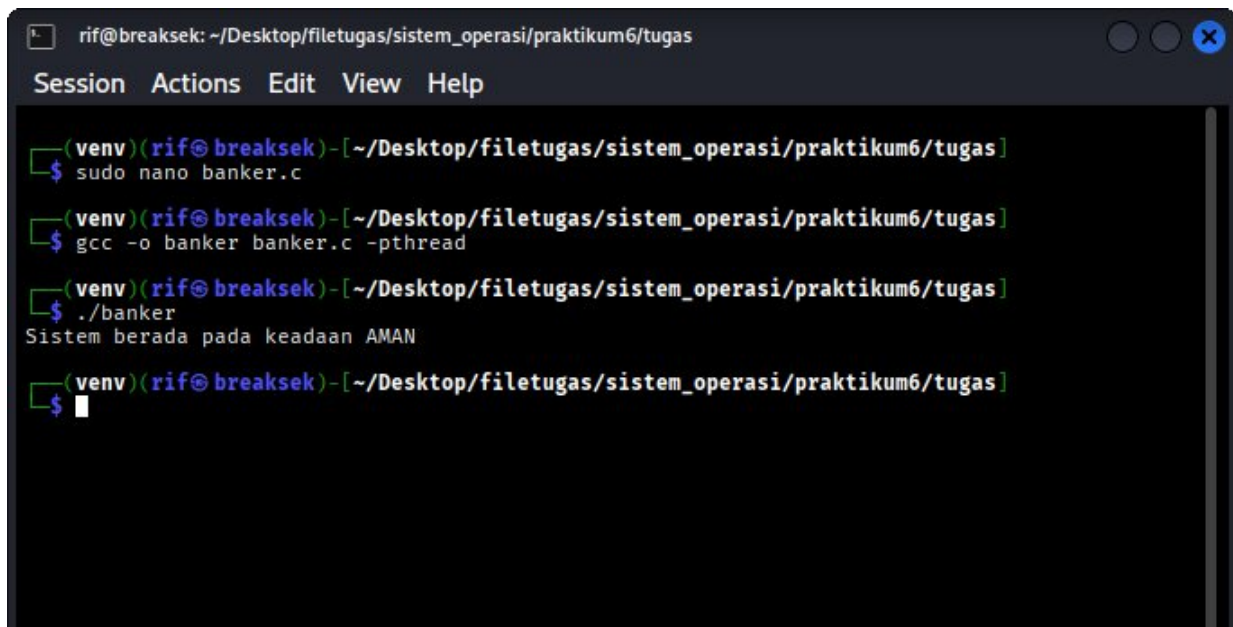
    if(!found) return 0;
}

return 1;
}

```

```
int main(){
    calc_need();
    if(is_safe()) printf("Sistem berada pada keadaan AMAN\n");
    else printf("Sistem TIDAK aman\n");
}
```

- Compile dan jalankan program



The screenshot shows a terminal window with the title bar "rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas". The window has a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal content shows the following commands and output:

```
(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano banker.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o banker banker.c -pthread

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./banker
Sistem berada pada keadaan AMAN

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$
```

6. TUGAS 4 — SIMULASI DEADLOCK TRANSFER BANK

a. Deskripsi

Pada tugas ini dilakukan simulasi deadlock dengan kasus dua orang yang melakukan transfer antar rekening secara bersamaan:

- Orang A: Rekening 1 → Rekening 2
- Orang B: Rekening 2 → Rekening 1

Jika kedua proses mengunci rekening secara terbalik, maka dapat terjadi deadlock.

b. Program Simulasi Deadlock

- Buat file dengan nama `transfer_deadlock.c` dan masukkan kode berikut

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int saldo;
    pthread_mutex_t lock;
} Account;

Account acc1 = {1, 1000, PTHREAD_MUTEX_INITIALIZER};
Account acc2 = {2, 1000, PTHREAD_MUTEX_INITIALIZER};

void transfer(Account *from, Account *to, int jumlah, const char *nama) {
    pthread_mutex_lock(&from->lock);
    printf("%s mengunci Rekening %d\n", nama, from->id);
    sleep(1); // memberi waktu thread lain mengunci rekening berbeda

    pthread_mutex_lock(&to->lock); // DEADLOCK terjadi di sini
    printf("%s mengunci Rekening %d\n", nama, to->id);

    from->saldo -= jumlah;
```

```

to->saldo += jumlah;

pthread_mutex_unlock(&to->lock);
pthread_mutex_unlock(&from->lock);
}

void *orangA(void *arg) {
    transfer(&acc1, &acc2, 100, "Orang A");
    return NULL;
}

void *orangB(void *arg) {
    transfer(&acc2, &acc1, 200, "Orang B");
    return NULL;
}

int main() {
    pthread_t A, B;
    pthread_create(&A, NULL, orangA, NULL);
    pthread_create(&B, NULL, orangB, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);
    return 0;
}

```

- Compile dan jalankan program


```
rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas
Session Actions Edit View Help

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano transfer_deadlock.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o transfer_deadlock transfer_deadlock.c -lpthread

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./transfer_deadlock
Orang A mengunci Rekening 1
Orang B mengunci Rekening 2
█
```

- **Analisa :** Program mengalami deadlock.
Hal ini terlihat karena program berhenti setelah salah satu thread mengunci satu rekening dan menunggu rekening lain yang sedang dikunci thread lainnya.

c. Deadlock Prevention (Lock Ordering)

- Buat file dengan nama transfer_prevent.c dan isi kode berikut

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int id;
    int saldo;
    pthread_mutex_t lock;
} Account;

Account acc1 = {1, 1000, PTHREAD_MUTEX_INITIALIZER};
Account acc2 = {2, 1000, PTHREAD_MUTEX_INITIALIZER};

void transfer(Account *from, Account *to, int jumlah, const char *nama) {
    // Lock ordering: selalu kunci rekening dengan ID lebih kecil dulu
    Account *first = (from->id < to->id) ? from : to;
    Account *second = (from->id < to->id) ? to : from;

    pthread_mutex_lock(&first->lock);
```

```

pthread_mutex_lock(&second->lock);

from->saldo -= jumlah;
to->saldo += jumlah;

pthread_mutex_unlock(&second->lock);
pthread_mutex_unlock(&first->lock);

printf("%s transfer berhasil tanpa deadlock\n", nama);
}

void *orangA(void *arg) {
    transfer(&acc1, &acc2, 100, "Orang A");
    return NULL;
}

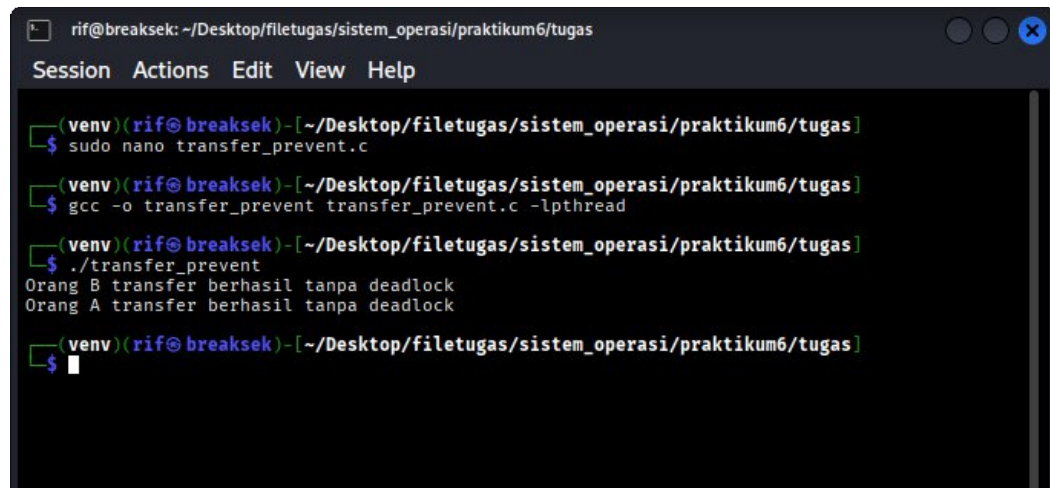
void *orangB(void *arg) {
    transfer(&acc2, &acc1, 200, "Orang B");
    return NULL;
}

int main() {
    pthread_t A, B;
    pthread_create(&A, NULL, orangA, NULL);
    pthread_create(&B, NULL, orangB, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);
    return 0;
}

```

- Compile dan jalankan program



```
rif@breaksek: ~/Desktop/filetugas/sistem_operasi/praktikum6/tugas
Session Actions Edit View Help

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ sudo nano transfer_prevent.c

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ gcc -o transfer_prevent transfer_prevent.c -lpthread

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$ ./transfer_prevent
Orang B transfer berhasil tanpa deadlock
Orang A transfer berhasil tanpa deadlock

(venv)(rif@breaksek)-[~/Desktop/filetugas/sistem_operasi/praktikum6/tugas]
$
```

- Analisa : Program tidak mengalami deadlock.
Teknik lock ordering menghilangkan kondisi circular wait sehingga kedua proses dapat selesai dengan normal.

7. KESIMPULAN

- Deadlock terjadi ketika dua thread mengunci resource secara silang.
- Keempat kondisi deadlock semuanya terpenuhi pada kasus ini.
- Lock ordering terbukti efektif mencegah deadlock tanpa overhead besar.
- Try-lock mencegah deadlock tetapi memerlukan retry loop yang kurang efisien.
- Banker's Algorithm dapat memastikan sistem berada dalam keadaan aman sebelum memberikan resource.
- Kasus transfer bank dua arah menunjukkan contoh nyata deadlock pada sistem keuangan.